

DAILY IT TRAINING

AWS Cloud Monitoring Project

Interview Explanation Guide for Cloud Support, NOC, and Junior SysAdmin Roles

Area	Cloud Support / NOC / Network Support Portfolio
Subtopic	AWS VPC, EC2, CloudWatch, alarms, logs, SNS, dashboards, troubleshooting, co
Lesson Date	2026-09-21

What's Inside

- How I Would Introduce This Project
- Why This Project Matters for Jobs
- Important Cost and Safety Notice
- Project Architecture
- What You Will Build
- Skills You Will Prove
- Prerequisites
- Step 1: Create a Billing Alarm First
- Step 2: Create the VPC
- Step 3: Internet Gateway and Route Table
- Step 4: Create Security Group
- Step 5: Create IAM Role for EC2 Monitoring
- Step 6: Launch EC2 Instance
- Step 7: Verify EC2 Health
- Step 8: Create SNS Email Topic

1. How I Would Introduce This Project

This project is an AWS Cloud Monitoring portfolio lab. The goal is to show that I can deploy a small cloud environment, monitor it, detect incidents, create alarms, review logs, and document troubleshooting steps like a Cloud Support Analyst or NOC Analyst.

In the lab I build a small AWS environment with a VPC, public subnet, EC2 Linux server, security group, CloudWatch metrics, CloudWatch alarms, a dashboard, log collection, SNS email alerting, and an incident runbook. I also include cost-control steps so the lab can be safely cleaned up after testing.

Interview pitch:

> I built an AWS monitoring lab to practice real cloud support work. I deployed a small EC2-based environment, configured monitoring using Amazon CloudWatch, created CPU/status-check alarms, collected logs, built a dashboard, and documented incident response steps. The project proves I can monitor cloud resources, troubleshoot common infrastructure issues, and communicate findings clearly through runbooks and GitHub documentation.

2. Why This Project Matters for Jobs

This lab connects directly to roles like:

- Cloud Support Associate
- NOC Analyst
- IT Support Analyst with cloud exposure
- Junior SysAdmin
- Infrastructure Support Analyst
- Junior DevOps / Platform Support
- Aviation IT cloud support
- Banking or enterprise cloud operations support

Employers want to see that you can do more than create a server. They want evidence that you understand operations:

- Is the server healthy?
- Are CPU, memory, disk, and network usage normal?
- Can alerts reach the support team?
- Can you check logs?
- Can you explain what happened during an incident?
- Can you clean up resources to avoid cost?

This lab proves those skills.

3. Important Cost and Safety Notice

This lab can create AWS resources that may cost money. Do not deploy resources unless you are ready to monitor cost and delete them after testing.

Cost-safety rules:

1. Use AWS Free Tier where possible.
2. Use a small EC2 instance type such as `t2.micro` or `t3.micro` if eligible.
3. Use one region only, for example `ca-central-1` or `us-east-1`.
4. Set a billing alarm before experimenting.
5. Stop or terminate EC2 when finished.
6. Delete unused EBS volumes, Elastic IPs, NAT Gateways, and load balancers.
7. Do not create a NAT Gateway for this beginner version because NAT Gateway can create charges.
8. Do not upload secrets, access keys, or private SSH keys to GitHub.

Portfolio wording:

> This project is a controlled training lab. It uses small AWS resources and includes cleanup steps to avoid unexpected costs.

4. Project Architecture

Beginner architecture:

```

User / Admin Laptop
|
| SSH / HTTP test
v
Internet Gateway
|
AWS VPC: 10.20.0.0/16
|
Public Subnet: 10.20.1.0/24
|
Security Group
|
EC2 Linux Web Server
|
CloudWatch Agent / Metrics / Logs
|
CloudWatch Dashboard + Alarms
|
SNS Email Alert

```

Services used:

| Service | Purpose |

|---|---|

| VPC | Private cloud network |

Subnet	Segment for EC2 server
Internet Gateway	Internet access for public subnet
Route Table	Sends internet traffic to gateway
Security Group	Firewall around EC2
EC2	Linux server to monitor
CloudWatch Metrics	CPU, status check, network, disk/memory if agent installed
CloudWatch Logs	Collect application/system logs
CloudWatch Alarms	Trigger alerts when threshold is breached
CloudWatch Dashboard	Visual monitoring page
SNS	Email notification for alarms
IAM Role	Allows EC2 to send logs/metrics to CloudWatch

5. What You Will Build

You will build:

1. A simple AWS network.
2. One Linux EC2 instance.
3. A security group allowing SSH from your IP and HTTP for testing.
4. A simple web page to generate traffic.
5. CloudWatch alarms for EC2 health and CPU.
6. CloudWatch dashboard.
7. Optional CloudWatch Agent for memory/disk/logs.
8. SNS email notification.
9. Incident documentation.
10. GitHub portfolio README.

6. Skills You Will Prove

By completing this lab, you can say you practiced:

- AWS VPC basics
- EC2 deployment
- Linux administration basics
- Security group configuration
- IAM role usage
- CloudWatch monitoring
- CloudWatch alarms
- SNS alerting
- Log collection

- Incident response
- Cost control
- Cloud troubleshooting documentation
- GitHub portfolio documentation

7. Prerequisites

You need:

- AWS account
- AWS Free Tier awareness
- IAM user or IAM Identity Center access
- MFA enabled on root account
- AWS CLI installed locally if using CLI
- SSH client
- GitHub account
- Basic Linux command-line comfort

If using the AWS Console only, you can still complete the lab. If using CLI, confirm identity:

```
aws sts get-caller-identity
```

Expected result:

```
{
  "UserId": "...",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/your-user"
}
```

8. Step 1: Create a Billing Alarm First

Before building anything, protect yourself from surprise cost.

Console path:

```
Billing and Cost Management -> Budgets -> Create budget
```

Recommended beginner budget:

```
Budget type: Cost budget
Amount: $5 or $10 monthly
Alert threshold: 50%, 80%, 100%
Email: your email address
```

Why this matters:

Cloud support teams must care about cost. This is also a good interview point because it shows operational maturity.

Interview explanation:

> Before deploying resources, I created a billing budget and alert. This shows that I understand cloud cost control and responsible lab management.

9. Step 2: Create the VPC

VPC design:

```
VPC Name: aws-monitoring-lab-vpc
CIDR: 10.20.0.0/16
Public Subnet: 10.20.1.0/24
Availability Zone: choose one AZ in your region
```

Console path:

```
VPC -> Your VPCs -> Create VPC
```

Use:

```
Name tag: aws-monitoring-lab-vpc
IPv4 CIDR: 10.20.0.0/16
```

Create subnet:

```
Name: aws-monitoring-public-subnet
CIDR: 10.20.1.0/24
```

Enable auto-assign public IPv4 on the subnet:

```
Subnet -> Actions -> Edit subnet settings -> Enable auto-assign public IPv4
```

10. Step 3: Internet Gateway and Route Table

Create Internet Gateway:

```
Name: aws-monitoring-igw
Attach to: aws-monitoring-lab-vpc
```

Create route table:

```
Name: aws-monitoring-public-rt
VPC: aws-monitoring-lab-vpc
```

Add route:

```
Destination: 0.0.0.0/0
Target: Internet Gateway aws-monitoring-igw
```

Associate the route table with your public subnet.

Verification concept:

If the subnet has a route to the Internet Gateway and the EC2 has a public IP, the server can be reached from the internet if the security group allows it.

11. Step 4: Create Security Group

Security group name:

```
aws-monitoring-web-sg
```

Inbound rules:

Type	Port	Source	Purpose
SSH	22	Your public IP only	Admin access
HTTP	80	0.0.0.0/0	Web test page
ICMP	Echo Request	Your public IP only	Optional ping test

Do not open SSH to the whole internet.

Bad rule:

```
SSH 22 from 0.0.0.0/0
```

Good rule:

```
SSH 22 from YOUR_PUBLIC_IP/32
```

Interview explanation:

> I restricted SSH access to my own IP and only opened HTTP publicly for testing. This follows least privilege.

12. Step 5: Create IAM Role for EC2 Monitoring

Create IAM role:

```
Role name: aws-monitoring-ec2-role
Trusted entity: EC2
```

Attach managed policies:

```
CloudWatchAgentServerPolicy
AmazonSSMManagedInstanceCore
```

Why:

- CloudWatchAgentServerPolicy lets the EC2 instance publish CloudWatch agent metrics and logs.
- AmazonSSMManagedInstanceCore allows Systems Manager access if configured, reducing SSH dependency.

13. Step 6: Launch EC2 Instance

Recommended settings:

```
Name: aws-monitoring-web01
AMI: Amazon Linux 2023 or Ubuntu LTS
Instance type: t2.micro or t3.micro
Network: aws-monitoring-lab-vpc
Subnet: aws-monitoring-public-subnet
Auto-assign public IP: enabled
Security group: aws-monitoring-web-sg
IAM role: aws-monitoring-ec2-role
Storage: 8 GB gp3
```

User data for a simple web server on Amazon Linux:

```
#!/bin/bash
dnf update -y
dnf install -y httpd stress-ng amazon-cloudwatch-agent
systemctl enable httpd
systemctl start httpd
cat > /var/www/html/index.html <<'EOF'
<html>
<head><title>AWS Cloud Monitoring Lab</title></head>
<body>
<h1>AWS Cloud Monitoring Lab</h1>
<p>Server: aws-monitoring-web01</p>
<p>Status: Running</p>
</body>
</html>
EOF
```

After launch, test in browser:

```
http://EC2_PUBLIC_IP
```

Expected result:

```
AWS Cloud Monitoring Lab
Server: aws-monitoring-web01
Status: Running
```

14. Step 7: Verify EC2 Health

AWS Console:

```
EC2 -> Instances -> select instance -> Status checks
```

Expected:

```
2/2 checks passed
```

Linux SSH test:

```
ssh -i your-key.pem ec2-user@EC2_PUBLIC_IP
```

Basic commands:

```
hostname
uptime
free -h
df -h
curl -I http://localhost
systemctl status httpd
```

Expected ideas:

- hostname returns EC2 host name
- uptime shows server running
- free shows memory
- df shows disk usage
- curl returns HTTP 200 or similar
- httpd is active/running

15. Step 8: Create SNS Email Topic

Console path:

```
SNS -> Topics -> Create topic
```

Settings:

```
Type: Standard
Name: aws-monitoring-alerts
```

Create subscription:

```
Protocol: Email
Endpoint: your email address
```

Important

Check your email and confirm the subscription. Alarms will not send notifications until you confirm.

16. Step 9: Create CloudWatch CPU Alarm

Console path:

```
CloudWatch -> Alarms -> Create alarm
```

Metric:

```
EC2 -> Per-Instance Metrics -> CPUUtilization  
Instance: aws-monitoring-web01
```

Threshold:

```
Statistic: Average  
Period: 5 minutes  
Condition: Greater than 70%
```

Action:

```
Send notification to SNS topic: aws-monitoring-alerts
```

Alarm name:

```
aws-monitoring-web01-high-cpu
```

Interview explanation:

> I created a CPU alarm to simulate a NOC-style alert. The alarm sends a notification through SNS so an operator can investigate.

17. Step 10: Create EC2 Status Check Alarm

Create another alarm for EC2 status checks.

Metric:

```
EC2 -> Per-Instance Metrics -> StatusCheckFailed
```

Threshold:

```
StatusCheckFailed >= 1 for 1 datapoint
```

Alarm name:

```
aws-monitoring-web01-status-check-failed
```

Why this matters:

Status check failures can indicate AWS infrastructure issue, OS issue, network problem, or instance-level failure.

18. Step 11: Optional CloudWatch Agent for Memory and Disk

Default EC2 metrics do not include memory and disk usage. To monitor memory and disk, install and configure the CloudWatch Agent.

On Amazon Linux:

```
sudo dnf install -y amazon-cloudwatch-agent
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
```

Simpler config file example:

```
{
  "metrics": {
    "append_dimensions": {
      "InstanceId": "${aws:InstanceId}"
    },
    "metrics_collected": {
      "mem": {
        "measurement": ["mem_used_percent"],
        "metrics_collection_interval": 60
      },
      "disk": {
        "measurement": ["used_percent"],
        "metrics_collection_interval": 60,
        "resources": ["*"]
      }
    }
  },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/httpd/access_log",
            "log_group_name": "/aws/ec2/aws-monitoring-web01/httpd/access",
            "log_stream_name": "${instance_id}"
          },
          {
            "file_path": "/var/log/httpd/error_log",
            "log_group_name": "/aws/ec2/aws-monitoring-web01/httpd/error",
            "log_stream_name": "${instance_id}"
          }
        ]
      }
    }
  }
}
```

Save it as:

```
sudo nano /opt/aws/amazon-cloudwatch-agent/bin/config.json
```

Start agent:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-a fetch-config \
-m ec2 \
-c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json \
-s
```

Check status:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -m ec2 -a status
```

Expected:

```
status: running
```

19. Step 12: Create CloudWatch Dashboard

Console path:

```
CloudWatch -> Dashboards -> Create dashboard
```

Dashboard name:

```
aws-monitoring-lab-dashboard
```

Add widgets:

- EC2 CPU utilization
- EC2 network in/out
- EC2 status checks
- Memory used percent, if CloudWatch Agent is configured
- Disk used percent, if CloudWatch Agent is configured
- Alarm status widget
- Log insights query widget, optional

Good dashboard story:

> The dashboard gives a single view of server health, performance, and alarm status, similar to what a NOC or cloud support team monitors.

20. Step 13: Generate a Test Incident

Only do this if you are comfortable. The goal is to trigger the CPU alarm.

Install stress tool if not installed:

```
sudo dnf install -y stress-ng
```

Generate CPU load:

```
stress-ng --cpu 2 --timeout 600s
```

Watch alarm:

```
CloudWatch -> Alarms -> aws-monitoring-web01-high-cpu
```

Expected flow:

```
OK -> INSUFFICIENT_DATA -> ALARM -> OK
```

You should receive an SNS email alert if your subscription is confirmed.

Incident note example:

```
Incident: High CPU utilization on aws-monitoring-web01
Detected by: CloudWatch alarm
Metric: CPUUtilization > 70%
Impact: Web server may respond slowly
Investigation: Checked uptime, top, CloudWatch metrics, service status
Cause: Simulated CPU stress test
Resolution: Stopped stress-ng process
Prevention: Add runbook and dashboard monitoring
```

21. Step 14: CloudWatch Logs Check

If CloudWatch Agent is configured, check logs:

```
CloudWatch -> Logs -> Log groups
```

Expected log groups:

```
/aws/ec2/aws-monitoring-web01/httpd/access
/aws/ec2/aws-monitoring-web01/httpd/error
```

Generate web traffic:

```
curl http://localhost
curl http://EC2_PUBLIC_IP
```

You should see access log events appear in CloudWatch Logs.

Log Insights example query:

```
fields @timestamp, @message
| sort @timestamp desc
| limit 20
```

22. Verification Checklist

Use this checklist before calling the project complete.

| Check | Expected result |

|---|---|

| Billing budget created | Budget alert configured |

| VPC created | 10.20.0.0/16 exists |

| Public subnet created | 10.20.1.0/24 exists |

| Internet Gateway attached | VPC has internet path |

| Route table configured | 0.0.0.0/0 points to IGW |

| Security group configured | SSH from your IP, HTTP open for test |

| EC2 running | Instance status running |

| EC2 status checks | 2/2 checks passed |

| Web page works | Browser shows lab page |

| IAM role attached | EC2 role visible on instance |

| SNS topic created | Email subscription confirmed |

| CPU alarm created | Alarm visible in CloudWatch |

| Status alarm created | Alarm visible in CloudWatch |

| Dashboard created | Metrics displayed |

| CloudWatch Agent optional | Memory/disk/logs visible |

| Incident test completed | CPU alarm triggered and documented |

| Cleanup complete | Resources stopped/deleted after lab |

23. Troubleshooting Scenario 1: Cannot SSH to EC2

Symptoms:

- SSH times out.
- Permission denied.
- Connection refused.

Check:

1. Is the instance running?
2. Does it have a public IPv4 address?
3. Is it in the public subnet?
4. Does the route table have 0.0.0.0/0 to the Internet Gateway?
5. Does the security group allow SSH from your current public IP?
6. Are you using the correct username?
7. Are key file permissions correct?

Commands:

```
chmod 400 your-key.pem  
ssh -i your-key.pem ec2-user@EC2_PUBLIC_IP
```

For Ubuntu AML:

```
ssh -i your-key.pem ubuntu@EC2_PUBLIC_IP
```

Interview answer:

> I would troubleshoot SSH by checking instance state, public IP, subnet route, security group source IP, network ACLs, username, and key permissions.

24. Troubleshooting Scenario 2: Website Not Loading

Symptoms:

- Browser cannot reach EC2 public IP.
- Connection times out.
- HTTP service is down.

Check:

```
sudo systemctl status httpd  
sudo ss -tulpn | grep :80  
curl -I http://localhost
```

AWS checks:

```
Security group allows TCP 80 from 0.0.0.0/0  
Subnet route table has internet route  
Instance has public IP  
Network ACL allows inbound/outbound traffic
```

Fix:

```
sudo systemctl enable httpd  
sudo systemctl restart httpd
```

25. Troubleshooting Scenario 3: Alarm Does Not Send Email

Symptoms:

- Alarm changes state but no email arrives.

Check:

1. SNS subscription confirmed?
2. Email in spam/junk folder?
3. Alarm action points to correct SNS topic?
4. Alarm entered ALARM state long enough?
5. Correct AWS region?

Interview answer:

> I would confirm the SNS subscription first, then verify alarm actions, region, metric threshold, and email filtering.

26. Troubleshooting Scenario 4: CloudWatch Agent Not Sending Metrics

Symptoms:

- CPU metrics show, but memory/disk metrics do not.
- Log groups are missing.

Check IAM role:

```
EC2 role must include CloudWatchAgentServerPolicy
```

Check agent status:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -m ec2 -a status
```

Check config syntax:

```
sudo cat /opt/aws/amazon-cloudwatch-agent/bin/config.json
```

Restart agent:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \  
-a fetch-config \  
-m ec2 \  
-c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json \  
-s
```

27. GitHub Repository Structure

Recommended repository name:

```
aws-cloud-monitoring-project
```

Recommended structure:

```
aws-cloud-monitoring-project/  
??? README.md  
??? architecture/  
?   ??? architecture-diagram.png  
??? screenshots/  
?   ??? 01-vpc.png  
?   ??? 02-ec2-running.png  
?   ??? 03-cloudwatch-dashboard.png  
?   ??? 04-cpu-alarm.png  
?   ??? 05-sns-email-alert.png  
?   ??? 06-logs.png  
??? runbooks/  
?   ??? ssh-troubleshooting.md  
?   ??? website-down.md  
?   ??? high-cpu-incident.md  
?   ??? cloudwatch-agent.md  
??? docs/  
?   ??? build-guide.md  
?   ??? verification-checklist.md  
?   ??? cost-cleanup.md  
??? evidence/  
??? incident-report-high-cpu.md
```

28. README Template

Use this in your GitHub README:

AWS Cloud Monitoring Project

Overview

This project demonstrates a small AWS monitoring environment using VPC, EC2, CloudWatch, SNS, dashboards, alarms, logs, and incident documentation.

Business Scenario

A company hosts a small web server in AWS. The IT/NOC team needs monitoring, alerting, and troubleshooting documentation to detect issues such as high CPU, failed status checks, service outages, and log errors.

Architecture

- VPC: 10.20.0.0/16
- Public subnet: 10.20.1.0/24
- EC2 Linux web server
- CloudWatch metrics and dashboard
- SNS email alerting
- Optional CloudWatch Agent for memory/disk/logs

Skills Demonstrated

- AWS networking basics
- EC2 support
- IAM role usage
- CloudWatch monitoring
- SNS alerts
- Linux troubleshooting
- Incident response documentation
- Cost-control awareness

Screenshots

Add screenshots of VPC, EC2, CloudWatch dashboard, alarms, SNS email, and logs.

Incident Example

Simulated high CPU using stress-ng and documented detection, investigation, and resolution.

29. Cleanup Steps

Very important. After finishing screenshots and documentation, clean up resources.

Delete or stop:

```
EC2 instance
EBS volumes not attached
Elastic IPs not needed
CloudWatch alarms
CloudWatch dashboards
CloudWatch log groups if no longer needed
SNS topic/subscription
Security group
Route table
Internet Gateway
Subnet
VPC
```

If you only stop EC2, storage may still cost money. Terminate the instance if you are done.

Cleanup checklist:

```
EC2 terminated
No unattached EBS volumes
No Elastic IP allocated
No NAT Gateway
No load balancer
CloudWatch logs deleted if not needed
Budget alert still active
```

30. Resume Bullets

Use these after completing the project:

- Built an AWS Cloud Monitoring portfolio lab using VPC, EC2, CloudWatch, SNS, IAM roles, dashboards, alarms, and Linux troubleshooting to simulate cloud operations support.
- Configured CloudWatch alarms for EC2 CPU utilization and status checks, with SNS email notifications and documented incident response for a simulated high-CPU event.
- Deployed and monitored a Linux web server in AWS, validating network access, security group rules, service health, logs, and dashboard metrics.
- Created operational runbooks for SSH access issues, website outage, CloudWatch alarm notification failures, and CloudWatch Agent troubleshooting.
- Documented cloud cost-control practices including AWS budgets, resource cleanup, and avoidance of high-cost resources in a beginner lab environment.

31. LinkedIn Featured Caption

I built an AWS Cloud Monitoring project to practice Cloud Support and NOC operations skills. The lab includes an AWS VPC, EC2 Linux web server, CloudWatch dashboard, CPU/status-check alarms, SNS email notifications, optional CloudWatch Agent logs/metrics, incident documentation, and cleanup/cost-control steps. This project helped me practice real cloud support tasks: monitoring, alerting, troubleshooting, documentation, and operational communication.

32. Interview Questions and Model Answers

Q1. What did you build?

I built an AWS Cloud Monitoring lab with a small VPC, public subnet, EC2 Linux web server, CloudWatch monitoring, alarms, SNS email alerts, and incident runbooks. The goal was to practice cloud operations support and monitoring.

Q2. Why did you use CloudWatch?

CloudWatch is AWS's native monitoring service. It collects metrics, logs, alarms, and dashboards so support teams can detect performance or availability issues.

Q3. What is the difference between a metric and a log?

A metric is numerical time-series data, such as CPU utilization or network packets. A log is text/event data, such as web server access logs or error logs. Metrics are good for thresholds and dashboards. Logs are good for investigation.

Q4. How would you troubleshoot an EC2 instance that is unreachable?

I would check the instance state, status checks, public IP, security group rules, subnet route table, internet gateway, network ACLs, key pair, username, and OS firewall/service status.

Q5. How did you control AWS costs?

I created a small lab using Free Tier-sized resources, configured a budget alert, avoided NAT Gateway and load balancers for the beginner version, and documented cleanup steps for EC2, EBS, Elastic IPs, log groups, and other resources.

33. Final Project Story for Hiring Managers

This AWS Cloud Monitoring project shows that I understand the operational side of cloud infrastructure. I did not only launch a server; I built a monitoring and alerting workflow around it. I configured CloudWatch alarms, SNS notifications, dashboard views, optional log collection, and runbooks for common incidents. This is directly related to Cloud Support, NOC, IT Support, and Junior SysAdmin work because these roles require monitoring systems, responding to alerts, checking logs, troubleshooting connectivity, and documenting resolution steps.

34. Next Project After This

After completing this AWS Cloud Monitoring Project, the best next lab is:

Secure Branch NAT/ACL/IPsec VPN Lab

Reason:

It connects your networking background with security and remote-site connectivity. Together, the Airport VLAN lab, AWS Monitoring lab, and Secure Branch VPN lab will give you a strong portfolio for NOC, network support, cloud support, and aviation IT roles.